School of Information and Computer Technology
Sirindhorn International Institute of Technology, Thammasat University
CSS326 Database Programming Laboratory

Laboratory Assignment #6: Structuring a Web App

Objectives:

- To restructure an Express app into a shared pool, routers and controllers.
- To validate request input and complete CRUD over HTTP with PUT and DELETE.
- To build a front-end page that lists, adds and deletes records with fetch().

Scenario: take your Lab 5 bookstore API and restructure it into a clean, maintainable web application: a shared connection pool, a router and a controller, then add validation, complete CRUD, and a front-end page. The database is the same bookstore from Lab 2, already filled in Lab 3.

```
books(     id, title, author, price, stock )
customers( id, name, email )
orders(    id, customer_id, book_id, quantity )

-- customer_id -> customers(id)
-- book_id     -> books(id)
```

Organize your project as shown below: db.js exports a mysql2 pool; routes/books.routes.js maps paths to controller functions and is mounted at /books in app.js; controllers/books.controller.js holds the logic and validation; and public/ holds the front-end. Use ? placeholders for every value. Submit your project files (you may zip them) together with screenshots of the API and the page. This assignment is marked out of 12.

```
project/
  app.js            db.js
  routes/books.routes.js
  controllers/books.controller.js
  public/index.html  public/main.js  public/style.css
```

1. Pool and structure. Create db.js exporting a mysql2 pool. Move your book routes into routes/books.routes.js (mounted at /books in app.js) and keep app.js lean. Include a screenshot showing GET /books still works. **(1.5 points)**
2. Controllers. Move the GET /books and GET /books/:id logic into controllers/books.controller.js; the route file should only reference the controller functions. GET /books/:id returns 404 when the id is unknown. **(1.5 points)**
3. Validate POST /books. In the create controller, reject the request with status 400 and a message if title is missing or price is not a positive number; otherwise insert the book with ? placeholders and return 201 with the new id. **(1.5 points)**
4. Update with PUT. Add PUT /books/:id that updates a book's price (and/or stock) from req.body using ? placeholders. Return 404 if no book has that id, otherwise report how many rows were updated. **(1.5 points)**

5. Delete with DELETE. Add DELETE /books/:id that removes a book by id. Return 404 if not found, otherwise reply 204 (or report affectedRows). **(1.5 points)**
6. Front-end: list and add. In public/, build index.html, main.js and a linked style.css that fetch and list all books, and add a new book through a form that sends a POST and then reloads the list. Basic styling (spacing, borders, a styled button) is enough. Include a screenshot of the page. **(2 points)**
7. Front-end: delete. Add a Delete button next to each book that sends a DELETE request and reloads the list. Include a screenshot showing a book removed. **(1.5 points)**
8. Short answer. In two or three sentences, explain one benefit of using a connection pool and one benefit of putting controllers in their own files. **(1 points)**